

HANSEL / GRETEL

배포 및 운영 문서

Deployment & Operation Guide

DRAFT — 최신 코드 기준 작성, 실물 end-to-end 검증 전

코드(origin/main) 검증 완료 · 실기기 통합 동작은 미검증

대상: Raspberry Pi 4 (Debian GNU/Linux 13 trixie) · Android 수신기 · Controller PC
기준 커밋: origin/main 351a1bf ("헤드가 명령 받고 node들에 detach 명령 분배") 반영
작성일: 2026-05-30

목차

목차	2
1. 시스템 개요	3
1-1. 구성 요소와 역할	3
1-2. 전체 구성도와 데이터 흐름	3
2. 네트워크 / IP 정책	5
2-1. 정적 IP 표 (운영).....	5
2-2. DHCP 동적 할당	5
2-3. 정적/동적 설정 스크립트	5
2-4. station_map.conf 와 자동분리의 관계	6
3. 포트맵 (Live 포트)	7
4. 영상 Relay 구조.....	9
4-1. 실행 파일 및 경로	9
4-2. hop별 송수신 (환경변수 기준).....	9
5. AP 및 부팅 실행 방식	11
5-A. 최초 설치 / 첫 부팅 준비 (새로 포맷한 Pi).....	11
5-B. 설치 완료 후 두 번째 부팅부터.....	11
6. 자동분리(Auto-detach) 구조	13
6-1. 실행 흐름 (Head 내부).....	13
6-2. Node 식별 / MAC.....	13
6-3. Latch 정책.....	13
6-4. 병렬 경로: Controller(keyboard.py)	13
6-5. 신규 경로: Head 중앙 detach 라우터 (커밋 351a1bf)	13
7. GPIO 핀 표 (최신 코드).....	15
7-1. Head Pi (Head_control.py).....	15
7-2. Node1 / Node2 / Node3 (동일).....	15
7-3. 이전 초안 대비 변경점	16
8. systemd · 로그 · 복구.....	17
8-1. systemd 유닛 / install 스크립트	17
8-2. 로그 확인	17
8-3. 장애 복구 빠른 표	17
9. 실물 검증 체크리스트 · 상태표	18

1. 시스템 개요

HANSEL/GRETEL은 1대의 Head와 3대의 Node로 구성된 분리형(detachable) 군집 로봇입니다. Head Pi가 Wi-Fi AP를 호스팅하고, 모든 Node와 Android 수신기가 이 AP에 접속합니다. Head는 CSI 카메라 영상을 H.264로 인코딩해 Node 체인을 거쳐 Android로 릴레이하고, Controller PC(또는 Head 자동분리 브리지)가 UDP로 각 유닛에 주행/서보/분리 명령을 보냅니다.

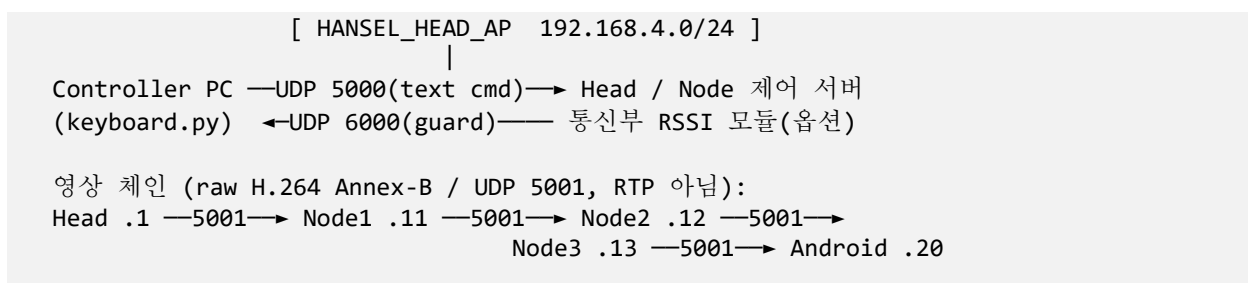
RSSI(신호세기) 저하가 지속되면 해당 Node를 물리적으로 분리(detach servo)하여 통신 범위를 벗어난 Node를 떼어내는 자동분리 기능을 갖습니다.

1-1. 구성 요소와 역할

유닛	역할	주요 프로세스 / 포트
Head Pi	Wi-Fi AP 호스트, 카메라 영상 송출, 주행/서보/분리 제어, RSSI 모니터 + 자동분리 브리지	start_ap.sh, Head_control.py(UDP 5000), head_h264_sender.py(UDP 5001), run_guard_detach.sh
Node1	주행/엔코더 PID + 분리 서보, 영상 릴레이(→Node2)	Node1_control.py(UDP 5000), udp_h264_relay.py(UDP 5001)
Node2	주행/엔코더 PID + 분리 서보, 영상 릴레이(→Node3)	Node2_control.py(UDP 5000), udp_h264_relay.py(UDP 5001)
Node3	주행/엔코더 PID + 분리 서보, 영상 릴레이(→Android)	Node3_control.py(UDP 5000), udp_h264_relay.py(UDP 5001)
Android	영상 수신/표시 (H.264 Annex-B 재조립)	android/ 앱, UDP 5001 수신
Controller PC	키보드 주행 제어 + RSSI guard 상태 수신	controller/desktop/keyboard.py (송신 UDP 5000, 수신 UDP 6000)

1-2. 전체 구성도와 데이터 흐름

Wi-Fi AP: HANSEL_HEAD_AP (192.168.4.0/24, 채널 6, WPA2-PSK)



자동분리 (Head 내부):

iw station dump → EWMA → 임계 → guard → detach_bridge
└UDP 5000 "detach_press"→ 대상 Node 제어 서버

데이터 평면 요약

제어 평면: UDP 5000 (텍스트 명령). 주행·서보·detach_press 모두 동일 포트.

영상 평면: UDP 5001 (raw H.264). Head→Node1→Node2→Node3→Android 단방향 릴레이.

감시 평면: Head 내부 RSSI 파이프라인 → 자동분리. Controller는 UDP 6000으로 guard 상태 수신 (옵션).

2. 네트워크 / IP 정책

운영 IP는 고정 정책을 사용합니다. 핵심 유닛은 정적 IP를, 임시 클라이언트(PC/실험 기기)는 DHCP 동적 풀을 사용합니다.

2-1. 정적 IP 표 (운영)

유닛	IP 주소	할당 방식	설정 위치
Head Pi (AP)	192.168.4.1/24	정적 (AP 게이트웨이)	ap/start_ap.sh
Node1	192.168.4.11	정적 (NetworkManager)	node1/net/setup_static_ip.sh
Node2	192.168.4.12	정적 (NetworkManager)	node2/net/setup_static_ip.sh
Node3	192.168.4.13	정적 (NetworkManager)	node3/net/setup_static_ip.sh
Android	192.168.4.20	정적 (폰에서 수동 설정)	Wi-Fi 고정 IP 수동 입력

게이트웨이/DNS: 192.168.4.1 (Head Pi). NAT는 현 단계에서 미설정 → 인터넷 경유 없음.

2-2. DHCP 동적 할당

항목	값	근거
동적 풀 범위	192.168.4.100 ~ 192.168.4.150	ap/dnsmasq.conf
임대 시간	12h	dhcp-range ... 12h
동적 할당 대상	AP에 붙는 임시 PC/실험 기기 등 (정적 미지정 클라이언트)	dnsmasq 자동 배정
정적 vs 동적 경계	정적 .1/.11/.12/.13/.20은 풀 (.100-.150) 밖 → 충돌 없음	dnsmasq.conf 주석 정책

주의: start_ap.sh 로그의 stale 텍스트

ap/start_ap.sh 의 마지막 요약 echo는 여전히 "DHCP: 192.168.4.10 - 192.168.4.50" 라고 출력합니다.

이는 화면 출력용 문구일 뿐이며, 실제 권위 있는 범위는 dnsmasq.conf의 .100-.150 입니다. 혼동 주의(코드 정리 권장).

2-3. 정적/동적 설정 스크립트

- **Head AP 주소(.1):** ap/start_ap.sh 가 wlan0 에 직접 할당 (DHCP 아님).
- **Node 정적 IP:** nodeN/net/setup_static_ip.sh --yes 가 nmcli 프로파일(ipv4.method manual)로 설정. autoconnect=yes.
- **Android:** 폰 Wi-Fi 설정에서 192.168.4.20 고정 IP 를 수동 입력 (dnsmasq 예약 아님).

2-4. station_map.conf 와 자동분리의 관계

자동분리 브리지(detach_bridge.py)는 RSSI가 저하된 station의 **wlan0 MAC**을 monitor/station_map.conf 에서 조회해 대상 Node IP로 변환합니다.

실기기 필수 작업

현재 station_map.conf 에는 실제 MAC이 없고 주석 예시만 있습니다.

실제 Node wlan0 MAC을 채우기 전에는 detach_bridge.py가 unknown_station ... action=skip 로그만 남기고 분리를 수행하지 않습니다.

실제 MAC 매핑값은 보안상 비공개 문서(PRIVATE_DEVICE_CONFIG)에 있으며, Head Pi 로컬에서만 반영합니다(저장소 커밋 금지).

3. 포트맵 (Live 포트)

아래는 최신 origin/main 코드에서 확인된 포트입니다. "실물 검증" 열은 코드 구조 확인과 실기기 통신 성공을 구분합니다.

기능	포트	Sender	Receiver/Listener	근거 파일
구동/서보 /분리 명령	UDP 5000	keyboard.py / detach_bridge.py	Head/Node*_control.py (0.0.0.0:5000)	*_control.py PORT=5000
detach command	UDP 5000	detach_bridge.py	대상 Node*_control.py	detach_bridge.py -- control-port 5000
영상 H.264 릴레이	UDP 5001	head_h264_sender.py / udp_h264_relay.py	Node relay / Android	VIDEO_PORT / RELAY*_PORT=5001
detach 요청 (신규 라우터)	UDP 6000	rss_i_to_head_detach_sender.py (통신부)	head_detach_router.py (Head 0.0.0.0:6000)	DETACH_ROUTER_PORT=6000
RSSI guard 상태	UDP 6000	통신부 RSSI 모듈 (외부)	keyboard.py GUARD_PORT	keyboard.py GUARD_PORT=6000
RSSI 실험 telemetry	UDP 5051	하류 이웃 Pi	상류 이웃 Pi	*_rss_i_listen.py TELEM_PORT
RSSI 실험 command	UDP 5052	상류 → 하류	이웃 Pi	*_rss_i_listen.py CMD_PORT
RSSI 실험 CSV file	TCP 5053	하류 → 상류	rss_i_monitor.py / 이웃	*_rss_i_listen.py FILE_PORT

포트 관련 정정 사항 (이전 초안 대비)

기능	포트	Sender	Receiver/Listener	근거 파일
----	----	--------	-------------------	-------

RSSI 실험 포트는 5051/5052/5053 입니다. 이전 초안의 6001/6002/6003은 최신 코드와 불일치하므로 폐기합니다.

UDP 5005는 최신 live 코드 어디에도 존재하지 않습니다 (stale 주석조차 없음).

rss_monitor/vid_quality_fps_logger.py 의 udp://@0.0.0.0:5000 은 별도 FFmpeg 화질 로거의 예시 입력일 뿐, live 제어 5000과 무관합니다.

systemd 자동 시작 대상: UDP 5000(control) · UDP 5001(camera/relay)만 부팅 서비스. 6000/5051/5052/5053은 수 · 실험용.

4. 영상 Relay 구조

4-1. 실행 파일 및 경로

역할	실행 파일	비고
Head 송신	head/camera/head_h264_sender.py (run_camera_stream.sh)	Picamera2 → H264 → UDP 청크 (1280×720@30, ~1Mbps, mtu 1200)
Node1 릴레이	node1/video/udp_h264_relay.py	byte-for-byte UDP 전달 (RTP/파싱 없음)
Node2 릴레이	node2/video/udp_h264_relay.py	동일
Node3 릴레이	node3/video/udp_h264_relay.py	최종 hop → Android
Android 수신	android/ 앱	바이트 누적 + Annex-B start code 재분할

빈 스텝 파일 주의 (live 경로 아님)

head/video/sender.sh, head/video/relay.sh, head/video/receiver.md, control/command_server.py, control/detach_trigger.sh 는 모두 빈 파일(스텝)입니다.

실제 live 영상 경로는 camera/head_h264_sender.py(Head) + video/udp_h264_relay.py(Node) 입니다.

4-2. hop 별 송수신 (환경변수 기준)

Hop	수신(listen)	송신 대상	환경변수 근거
Head → Node1	—	192.168.4.11:5001	hansel-head.env VIDEO_FIRST_HOP_IP
Node1 → Node2	0.0.0.0:5001	192.168.4.12:5001	hansel-node1.env RELAY_DST_IP
Node2 → Node3	0.0.0.0:5001	192.168.4.13:5001	hansel-node2.env RELAY_DST_IP
Node3 → Android	0.0.0.0:5001	192.168.4.20:5001	hansel-node3.env RELAY_DST_IP

부팅 자동 시작: Head는 hansel-head-camera.service, Node는 hansel-nodeN-relay.service 로 자동 기동(설치·enable 후).

검증 상태

코드상 연결 구조: 확인됨 (체인/포트/환경변수 일치).

실기기 end-to-end 영상 relay 성공: 미검증 → **실물 검증 필요**. Android 수신 표시까지 확인 전에는 "동작"으로 단정하지 않습니다.

5. AP 및 부팅 실행 방식

5-A. 최초 설치 / 첫 부팅 준비 (새로 포맷한 Pi)

1. **git 설치 및 저장소 clone:** `sudo apt install -y git && git clone <repo>`
2. **의존성 설치:** `python3, RPi.GPIO, picamera2(Head), hostapd, dnsmasq, iw, NetworkManager(nmcli)`
3. **systemd 유닛 설치/enable:** Head → `sudo ./systemd/install_head_services.sh --yes` ; Node → `sudo ./systemd/install_nodeN_services.sh --yes` (enable 만, AP 는 start 안 함)
4. **Node 정적 IP 적용:** `sudo bash net/setup_static_ip.sh --yes` (wlan0 가 HANSEL_HEAD_AP 에 정적 접속)
5. **Head 로컬 전용 설정:** `monitor/station_map.conf` 에 실제 Node MAC 반영 (비공개 문서 참조, 커밋 금지)
6. **최초 AP 기동:** `sudo systemctl start hansel-ap.service` (또는 `sudo bash ap/start_ap.sh --yes`)

△ SSH 끊김 경고 및 복구 준비

start_ap.sh 는 wlan0를 Wi-Fi 클라이언트 → AP 모드로 전환합니다. 현재 wlan0 Wi-Fi로 SSH 중 이면 세션이 끊깁니다.

최초 AP 기동 전에 콘솔/키보드·모니터 직접 접속 또는 유선 복구 경로를 준비하세요.

복구: 콘솔에서 `sudo bash ap/stop_ap.sh --yes` (이전 Wi-Fi 연결 `netplan-wlan0-ssing` 으로 복구).

AP 성공 기준: SSID HANSEL_HEAD_AP 가 보이고, wlan0=192.168.4.1, Node/PC가 접속·DHCP 또는 정적 join 성공, hostapd/dnsmasq active.

5-B. 설치 완료 후 두 번째 부팅부터

동작	설명
Head AP 자동 기동	<code>hansel-ap.service</code> 가 enable 되어 매 부팅 시 자동으로 start_ap.sh 실행 → AP 자동 상승 (최초 설치 시에만 수동 start 필요).
Head 제어/영상/모니터	<code>hansel-head-control / -camera / -monitor</code> 자동 시작 (After=hansel-ap)
Node AP 재접속	<code>autoconnect=yes</code> 프로필로 wlan0가 HANSEL_HEAD_AP에 자동 재접속 ; <code>control/relay</code> 서비스 자동 시작
Android	수동: 폰 Wi-Fi로 HANSEL_HEAD_AP 접속 + 수신 앱 실행 필

동작	설명
	요

부팅 후 최소 점검 명령

```
# Head
systemctl status hansel-ap.service hansel-head-control.service
ip addr show wlan0           # 192.168.4.1 확인
iw dev wlan0 station dump    # 접속한 Node MAC/신호 확인
journalctl -u hansel-head-camera.service -f
# Node
systemctl status hansel-node1-control.service hansel-node1-relay.service
ping -c2 192.168.4.1         # AP 도달 확인
```

종료 / 재시작 / 장애 복구

- **AP 중단·복구:** `sudo systemctl stop hansel-ap.service` 또는 `sudo bash ap/stop_ap.sh --yes`
- **서비스 재시작:** `sudo systemctl restart hansel-<unit>`
- **프로세스 강제 종료:** `sudo pkill -f Head_control.py / udp_h264_relay.py`
- **분리 latch 초기화(미션 전):** `sudo bash monitor/reset_detach_state.sh --yes && sudo systemctl restart hansel-head-monitor`

6. 자동분리(Auto-detach) 구조

6-1. 실행 흐름 (Head 내부)

```
rss_i_reader.sh   iw dev wlan0 station dump → station=<MAC> signal_dbm=<v>
|
link_filter.py    EWMA 평활 (alpha=0.3)
|
link_watch.py     임계 비교 (threshold=-70dBm) → link_degraded / link_ok
|
link_guard.py      연속 카운트 (degraded=10, recover=6, interval=0.2s)
                  → guard_status=detach_candidate / recovered / watching
↓
detach_bridge.py station_map.conf(MAC→IP) 조회
                  ↳ UDP "detach_press" → 대상 Node 제어 서버 :5000 (latch 후 1회)
```

통합 러너: run_guard_detach.sh = run_monitor.sh(파이프라인) | detach_bridge.py.
systemd hansel-head-monitor.service 로 부팅 자동 시작.

6-2. Node 식별 / MAC

- 분리 대상 식별자 = 저하된 station 의 wlan0 MAC. station_map.conf 에서 MAC→(IP, 이름)으로 매핑.
- 실제 MAC 은 Head Pi 로컬에서만 station_map.conf 에 기입(저장소 커밋 금지). 비공개 문서에 매핑값 수록.

6-3. Latch 정책

- 한 미션당 **Node 1 회만 분리**. 상태는 /var/lib/hansel/detach_state.json 에 영속 → 재시작/재부팅에도 재분리 안 함.
- RSSI 가 잠시 회복(recovered)해도 재무장 안 함. reset_detach_state.sh --yes (미션 전 수동)로만 해제.

6-4. 병렬 경로: Controller(keyboard.py)

Head 내부 브리지와 **별개**로, Controller의 keyboard.py 도 UDP 6000에서 통신부 guard 메시지(예 "NODE1,detach_candidate")를 받아 해당 유닛에 detach를 보내고 다음 유닛 주행을 비활성화하는 별도 자동분리 경로를 갖습니다.

6-5. 신규 경로: Head 중앙 detach 라우터 (커밋 351a1bf)

최신 커밋에서 통신부 → Head 라우터 → 액추에이터 유닛 의 중앙집중 detach 분배 경로가 추가되었습니다.

```
통신부 RSSI 판단
├─ rss_i_to_head_detach_sender.py request_detach_from_head("NODE2")
│   └─ UDP → HEAD_IP:6000
head_detach_router.py (Head Pi, listen UDP 6000)
```

└ 액추에이터 결정 → UDP "detach_press" → actor_ip:5000 (3회 반복)

기구 매핑(액추에이터): NODE1 분리→Head 서보, NODE2 분리→Node1 서보, NODE3 분리→Node2 서보. 분리 대상이 아니라 앞 유닛의 서보가 체결을 푼다는 점에 주의.

- 라우터 listen 포트: UDP 6000 (DETACH_ROUTER_PORT). 제어 송신: UDP 5000 (ROBOT_CONTROL_PORT).
- 기본 IP 는 임시 SSH(10.180.86.x) → AP 운용 시 HEAD_IP/NODE*_IP 환경변수를 192.168.4.x 로 지정해야 함.
- 동일 target 반복 분리 방지: ONE_SHOT_PER_TARGET=1, 쿨다운 3600s.
- systemd 미등록(현재 수동 실행). 부팅 자동 시작 대상 아님 — **실물 검증 필요.**

검증 상태 — 자동분리

코드 흐름: 확인됨 (RSSI→guard→bridge→UDP 5000→detach servo, 그리고 통신부→Head 라우터(6000)→actor:5000).

실기기 자동분리 성공: 미검증 → **실물 검증 필요.** station_map.conf 실제 MAC 반영 및 라우터 기동 후에야 동작합니다.

7. GPIO 핀 표 (최신 코드)

7-1. Head Pi (Head_control.py)

기능	BCM GPIO	물리 핀	상수명
메인 좌 모터 PWM (ENA)	18	12	ENA_PIN
메인 좌 모터 IN1	23	16	IN1_PIN
메인 좌 모터 IN2	24	18	IN2_PIN
메인 우 모터 PWM (ENB)	13	33	ENB_PIN
메인 우 모터 IN3	27	13	IN3_PIN
메인 우 모터 IN4	22	15	IN4_PIN
앞 DC모터 L ENA	12	32	FRONT_ENA_PIN
앞 DC모터 L IN1	4	7	FRONT_IN1_PIN
앞 DC모터 L IN2	25	22	FRONT_IN2_PIN
앞 DC모터 R ENB	19	35	FRONT_ENB_PIN
앞 DC모터 R IN3	5	29	FRONT_IN3_PIN
앞 DC모터 R IN4	7	26	FRONT_IN4_PIN
좌 엔코더 A	20	38	LEFT_ENC_A
좌 엔코더 B	21	40	LEFT_ENC_B
우 엔코더 A	16	36	RIGHT_ENC_A
우 엔코더 B	26	37	RIGHT_ENC_B
헤드 서보	17	11	HEAD_SERVO_PIN
분리 서보	6	31	DETACH_SERVO_PIN

핀 충돌 해소 확인 (최신 코드 기준)

이전 충돌(HEAD_SERVO_PIN 과 FRONT_ENA_PIN 이 GPIO12 공유)은 팀원 커밋 351a1bf 에서 **HEAD_SERVO_PIN** 을 **GPIO17(물리 11)**로 이동 하여 해소되었습니다.

현재 Head 18개 핀 모두 중복 없음 → **check_duplicate_pins()** 통과 예상.

최신 코드 기준 충돌 해소 확인, 실물 구동 검증 필요 (실기기에서 헤드 서보·앞모터 동시 동작 확인 전).

7-2. Node1 / Node2 / Node3 (동일)

기능	BCM GPIO	물리 핀	상수명
좌 모터 PWM (ENA)	18	12	ENA_PIN
좌 모터 IN1 / IN2	23 / 24	16 / 18	IN1_PIN/IN2_PIN
우 모터 PWM (ENB)	13	33	ENB_PIN
우 모터 IN3 / IN4	27 / 22	13 / 15	IN3_PIN/IN4_PIN
좌 엔코더 A / B	20 / 21	38 / 40	LEFT_ENC_A/B
우 엔코더 A / B	16 / 26	36 / 37	RIGHT_ENC_A/B
분리 서보	6	31	DETACH_SERVO_PIN

- Node 에는 헤드 서보·앞 DC 모터가 없어 핀 충돌이 없습니다 (check_duplicate_pins 통과).
- Node1/2/3 의 핀 정의는 완전히 동일합니다 (Node3 는 코드 구조만 다를 뿐 핀은 동일).

7-3. 이전 초안 대비 변경점

- 이전 초안은 분리 서보(GPIO6)만 표기 → 본 문서는 Head 의 메인/앞모터/엔코더/헤드서보 전체를 반영.
- 팀원 커밋 351a1bf 가 **HEAD_SERVO_PIN** 을 12→17 로 변경하여 GPIO12 충돌을 해소함. 본 문서 핀 표는 이를 반영함.

8. systemd · 로그 · 복구

8-1. systemd 유닛 / install 스크립트

유닛	실행	설치
hansel-ap.service	start_ap.sh --yes (oneshot, ExecStop=stop_ap.sh)	install_head_services.sh
hansel-head-control.service	Head_control.py (UDP 5000)	동일
hansel-head-camera.service	run_camera_stream.sh (UDP 5001)	동일
hansel-head-monitor.service	run_guard_detach.sh (RSSI+자동분리)	동일
hansel-nodeN-control.service	NodeN_control.py (UDP 5000)	install_nodeN_services.sh
hansel-nodeN-relay.service	run_video_relay.sh (UDP 5001)	동일

install 스크립트는 enable만 하고 AP/네트워크를 즉시 start 하지 않습니다 — 의도된 안전장치.

8-2. 로그 확인

```
journalctl -u hansel-head-control.service -f
journalctl -u hansel-head-monitor.service -f # 자동분리 이벤트
journalctl -u hansel-nodeN-relay.service -f # 영상 패킷 통계
tail -f raspberry/head/monitor/logs/monitor.log # --log 사용 시
```

8-3. 장애 복구 빠른 표

증상	조치
AP 전환 후 SSH 끊김	콘솔에서 sudo bash ap/stop_ap.sh --yes (Wi-Fi 클라이언트 복귀)
제어 무반응	systemctl status/restart hansel-*-control; ping 192.168.4.x
영상 안 나옴	Android Wi-Fi/IP(.20) 확인 → 각 relay journalctl 패킷 카운트 → Head camera 서비스 확인
분리 안 됨	station_map.conf MAC 반영 여부, detach_state.json latch, hansel-head-monitor 로그 확인
분리가 다시 안 됨(미션 재시작)	reset_detach_state.sh --yes 후 monitor 재시작

9. 실물 검증 체크리스트 · 상태표

항목	코드 확인	실물 검증
GPIO 핀 정의 / 충돌 점검	✓ 충돌 해소 (12→17)	필요(서보 구동)
정적/동적 IP 정책	✓	필요
UDP 5000 제어 명령 수신	✓	필요
영상 5001 체인 Head→...→Android	✓	필요
AP 자동 기동(2nd boot)	✓	필요
Node AP 자동 재접속	✓	필요
RSSI→자동분리(detach_press)	✓	필요(MAC 반영 후)
latch 1회 분리 / reset	✓	필요

문서 성격 재확인

본 문서는 최신 **origin/main(351a1bf)** 코드 기준으로 작성된 **DRAFT** 입니다.

표의 "실물 검증 필요" 항목은 실기기 end-to-end 테스트로 확인되기 전까지 "동작 확인됨"으로 간주하지 않습니다.